Southeast University

Introduction to Software Engineering

Wang Lulu，Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

Part 3. Quality Assurance

- **Chap 15-16. Software Quality**
- **Chap 17-19. Testing Strategy & Techniques**
- **Chap 20-21. Security Engineering & SCM**

Part 4. Software Project Management

Chap 17-19. Software Testing Strategies and Techniques

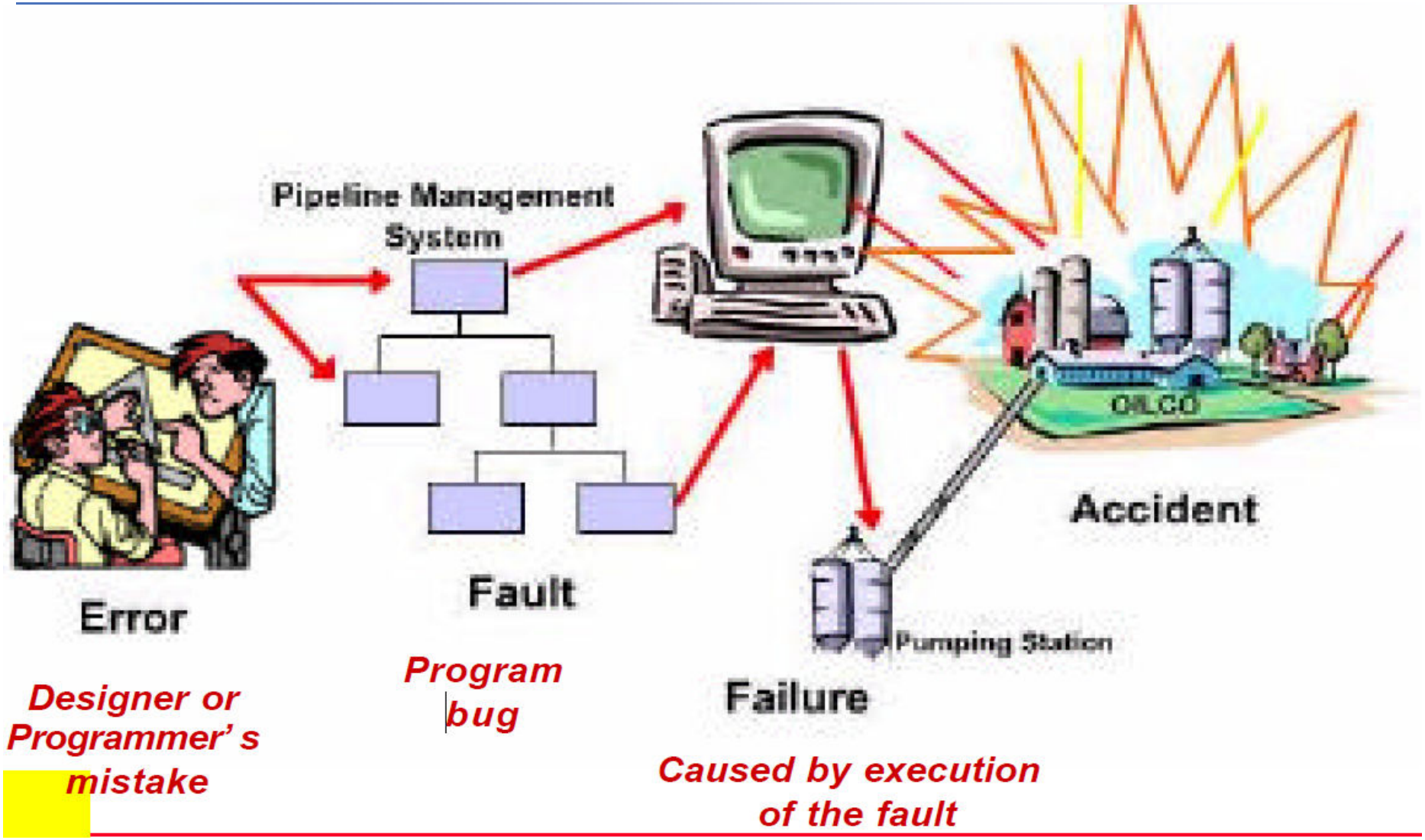
- 1. Concepts
- 2. V model vs. W model
  - Verification & Validation
- 3. Testing Strategies
  - Unit testing
  - Integration testing
  - System testing
  - Validation testing
  - Testing vs. Debug
- 4. Testing techniques
  - White box vs. Black box
  - Static testing vs. Dynamic testing

Error, Fault and Failure

- **Error**: a mistake in the design or programming; occurs when the system does not produce the expected output
- **Fault**: a mistake in the code; the result of an error
- **Failure**: the occurrence of a software fault

Users don't observe errors or faults. They observe execution failures.
------H. Mills

- Error：设计或编程中的错误；系统没有产生预期的输出
- Fault：代码中的错误；Error的结果
- Failure：软件错误发生

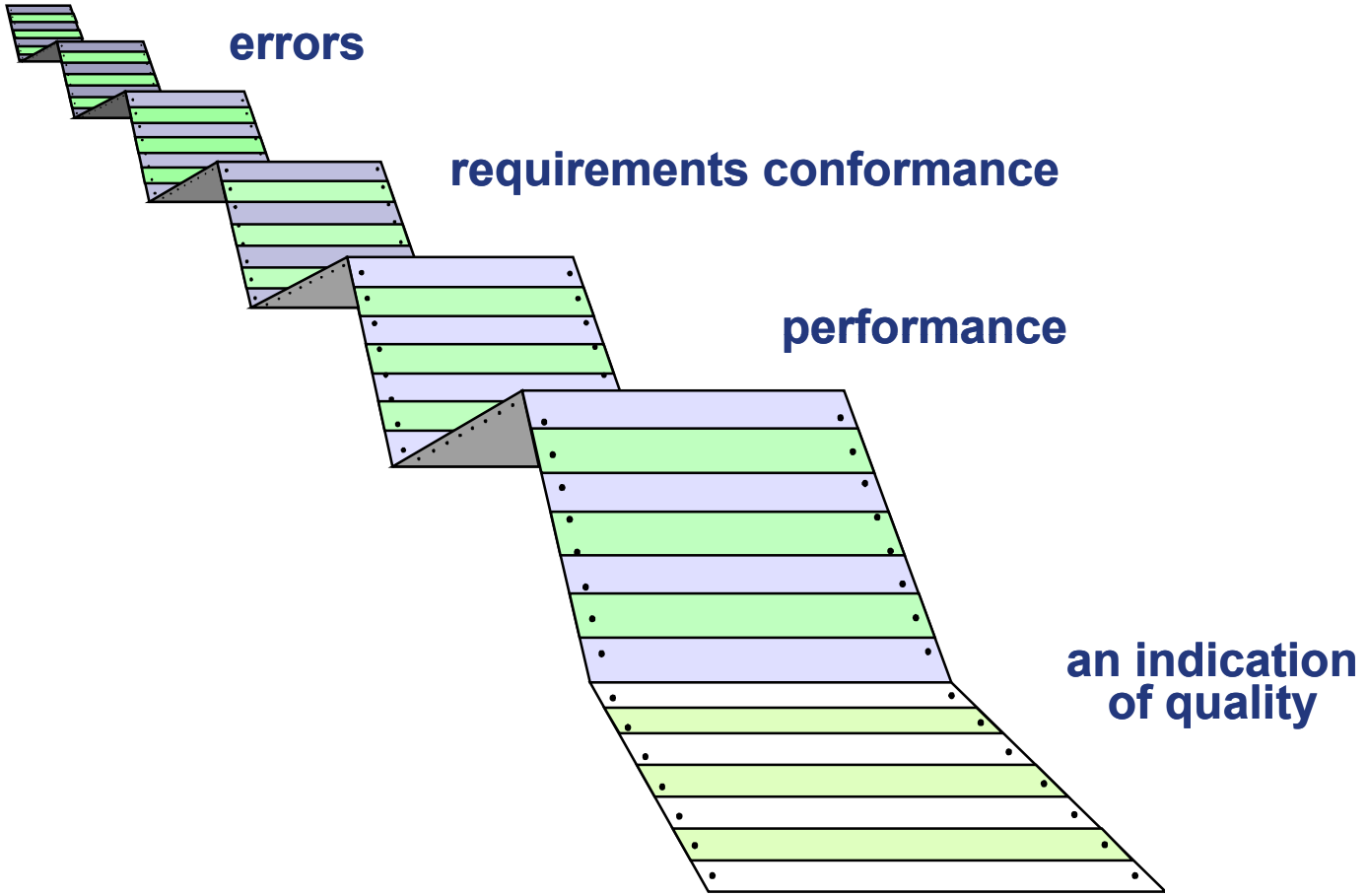

Pipeline Management System
OILCO
Accident
Error
Fault
Pumping Station
Failure
Designer or Programmer's mistake
Program bug
Caused by execution of the fault

Software Testing

Testing is the process of exercising a program with the specific intent of finding errors and faults prior to delivery to the end user.

测试是在交付给最终用户之前测试程序的过程，其特定目的是发现错误和故障。

What Testing Shows



Who Tests the Software?

developer

Understands the system
but, will test "gently"
and, is driven by "delivery"

了解系统，但会“温和地”进行测试，并由“交付”驱动

independent tester

Must learn about the system,
but, will attempt to break it
and, is driven by quality

必须了解这个系统，但是，会尝试打破它，并且由质量驱动

programming

SOFTWARE TESTER

Microsoft's leadership in software dates to the advent of the microcomputer in 1974. With over two million installations, our first product, Microsoft BASIC, is the most widely used language in the world. As part of the Microsoft team, you will share our commitment to product development and technical excellence. We currently are seeking Software Testers to work on our newest applications software products.

As a Software Tester, you will design, execute and document tests of application software. You will generate test scripts and automatic test packages. This is a challenging and highly visible position within a fast growing division of Microsoft.

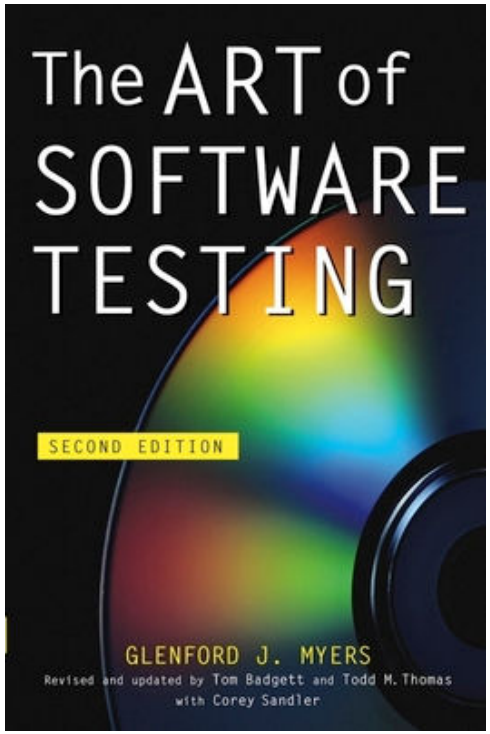
Requirements: background in math, computers, programming; preferably two years software test experience. Must be detail oriented, organized and have excellent written and oral skills.

Explore your future with Microsoft by sending a resume or letter that outlines your education, experience, and salary requirements to: **Microsoft Corporation, Dept. ST, 10700 Northup Way, Box 97200, Bellevue, WA 98009.** We are an equal opportunity employer.

MICROSOFT®

The High Performance Software™

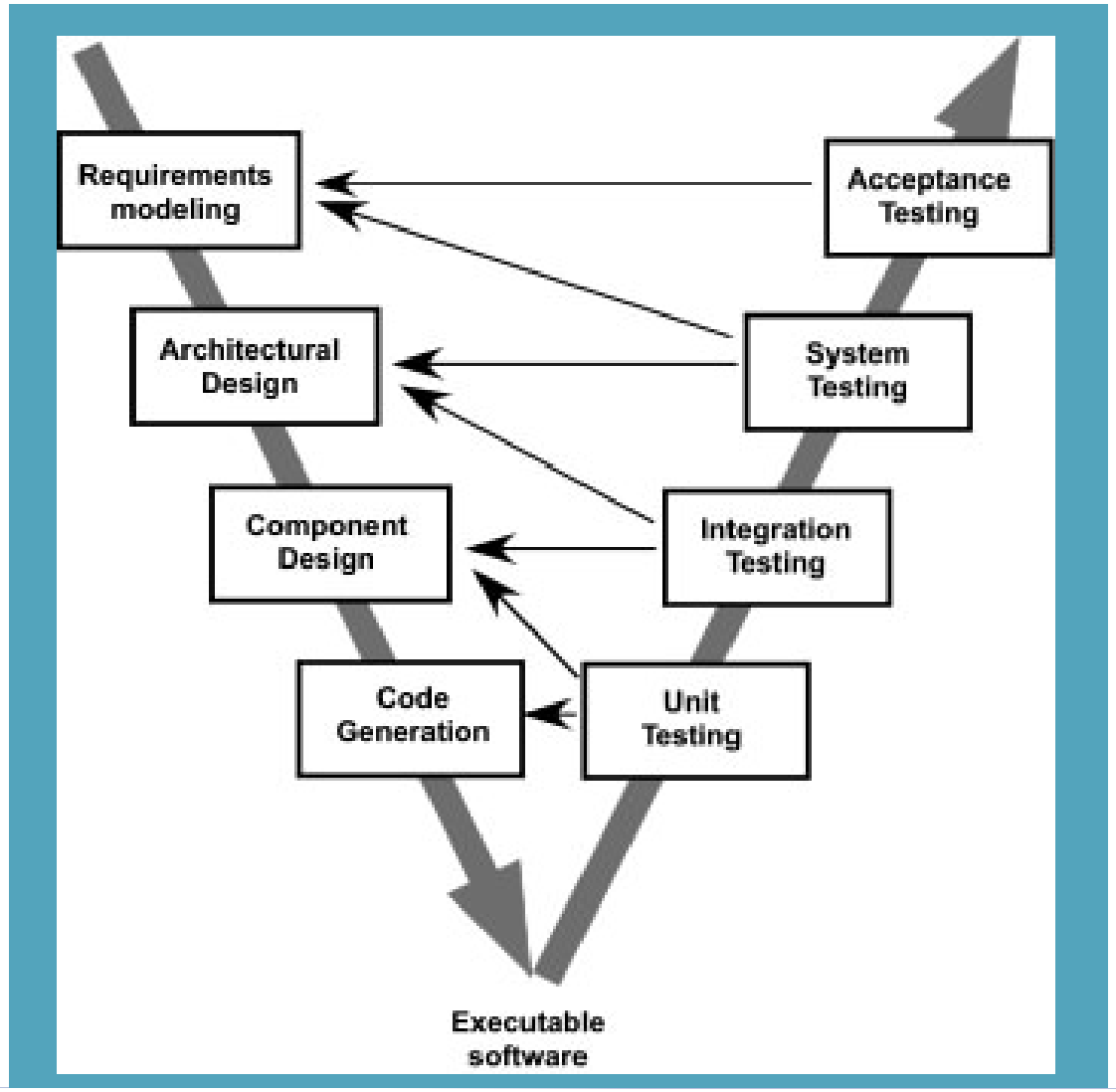
Figure 2-1: *Seattle Times* 1985 advertisement for software testers.



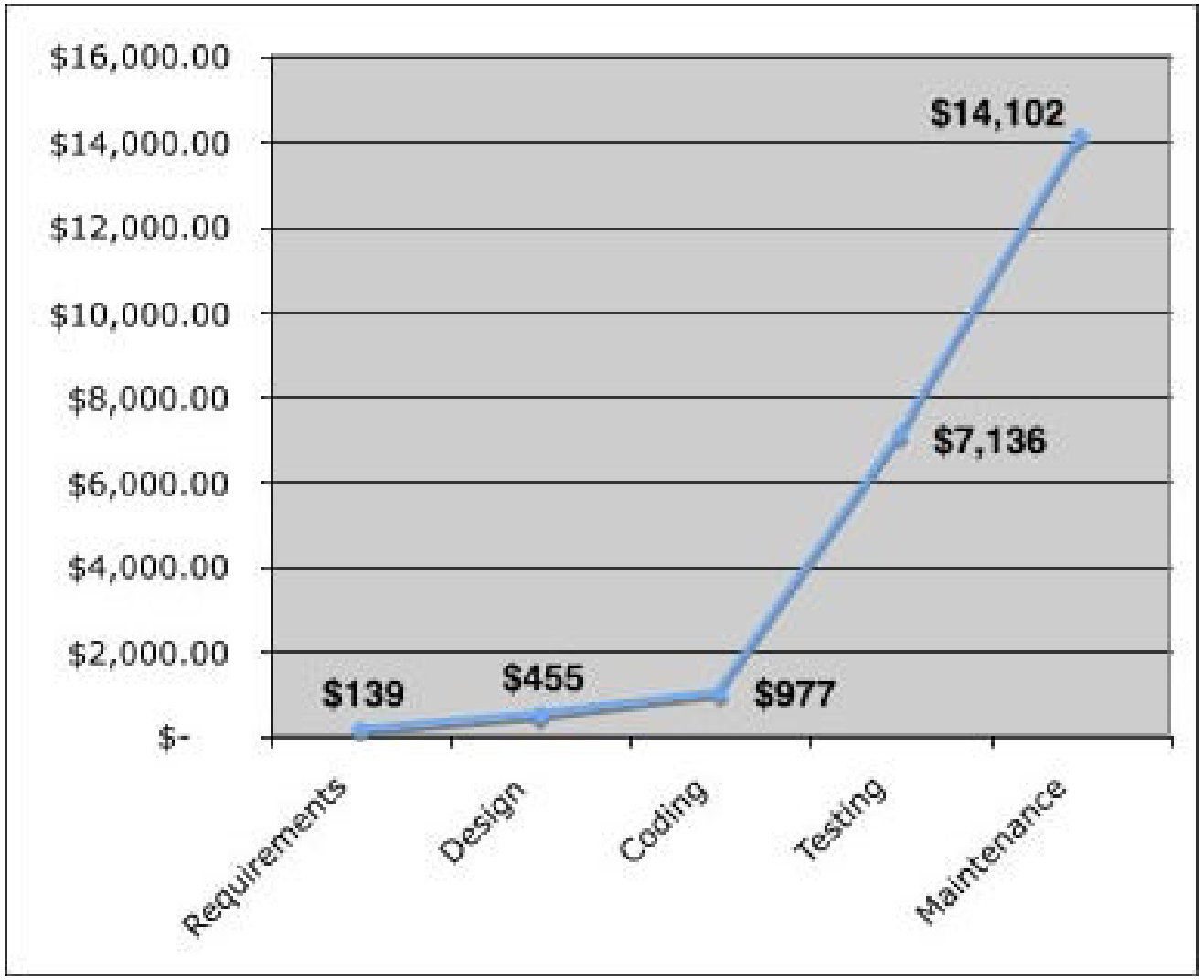
1979

Glenford J. Myers

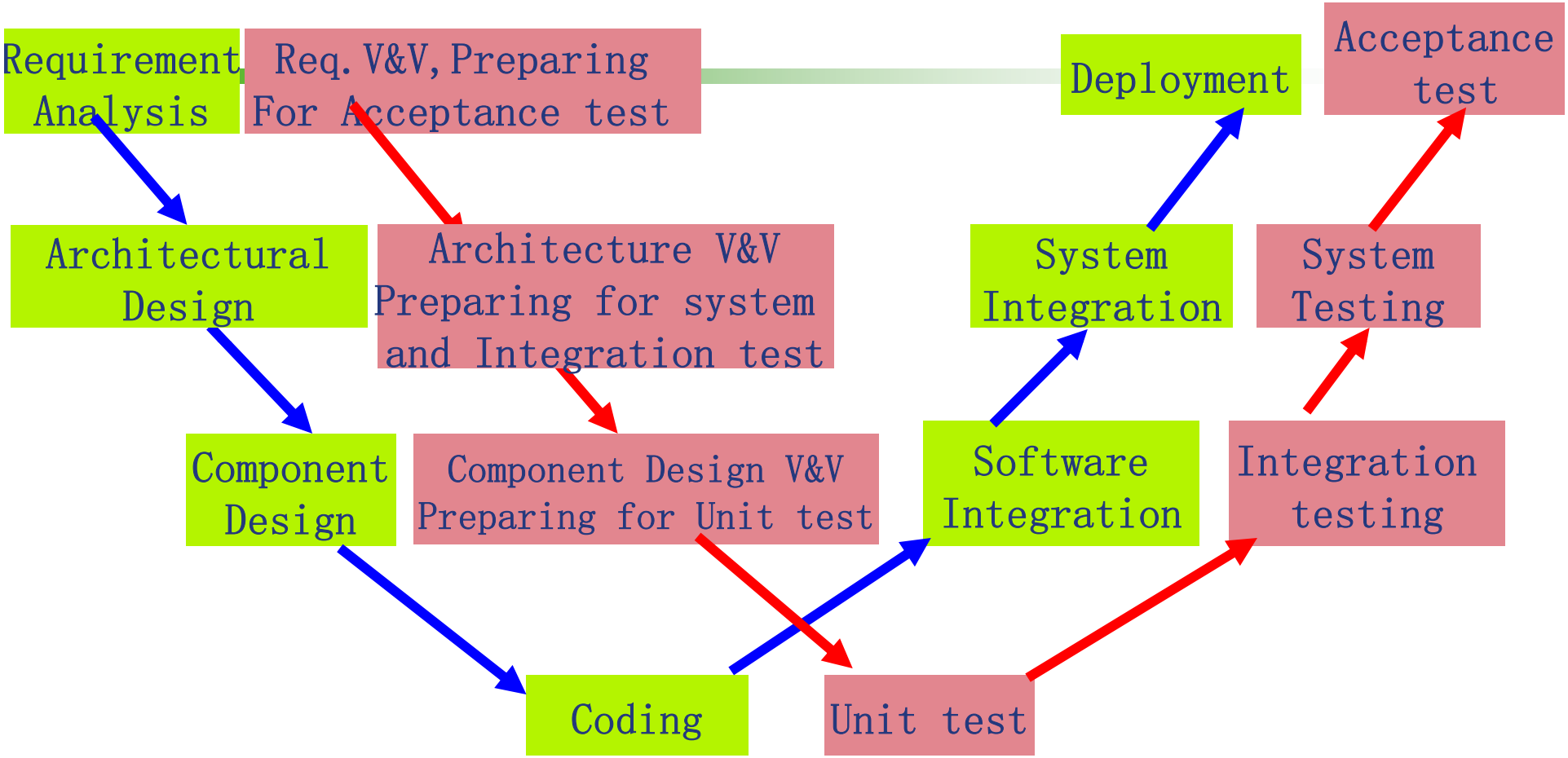
The V-Model


Requirements modeling
Architectural Design
Component Design
Code Generation
Unit Testing
Integration Testing
System Testing
Acceptance Testing
Executable software

The Cost of Quality



The W-Model


Requirement Analysis
Req. V&V, Preparing For Acceptance test
Deployment
Acceptance test
Architectural Design
Architecture V&V Preparing for system and Integration test
System Integration
System Testing
Component Design
Component Design V&V Preparing for Unit test
Software Integration
Integration testing
Coding
Unit test

Verification vs. Validation

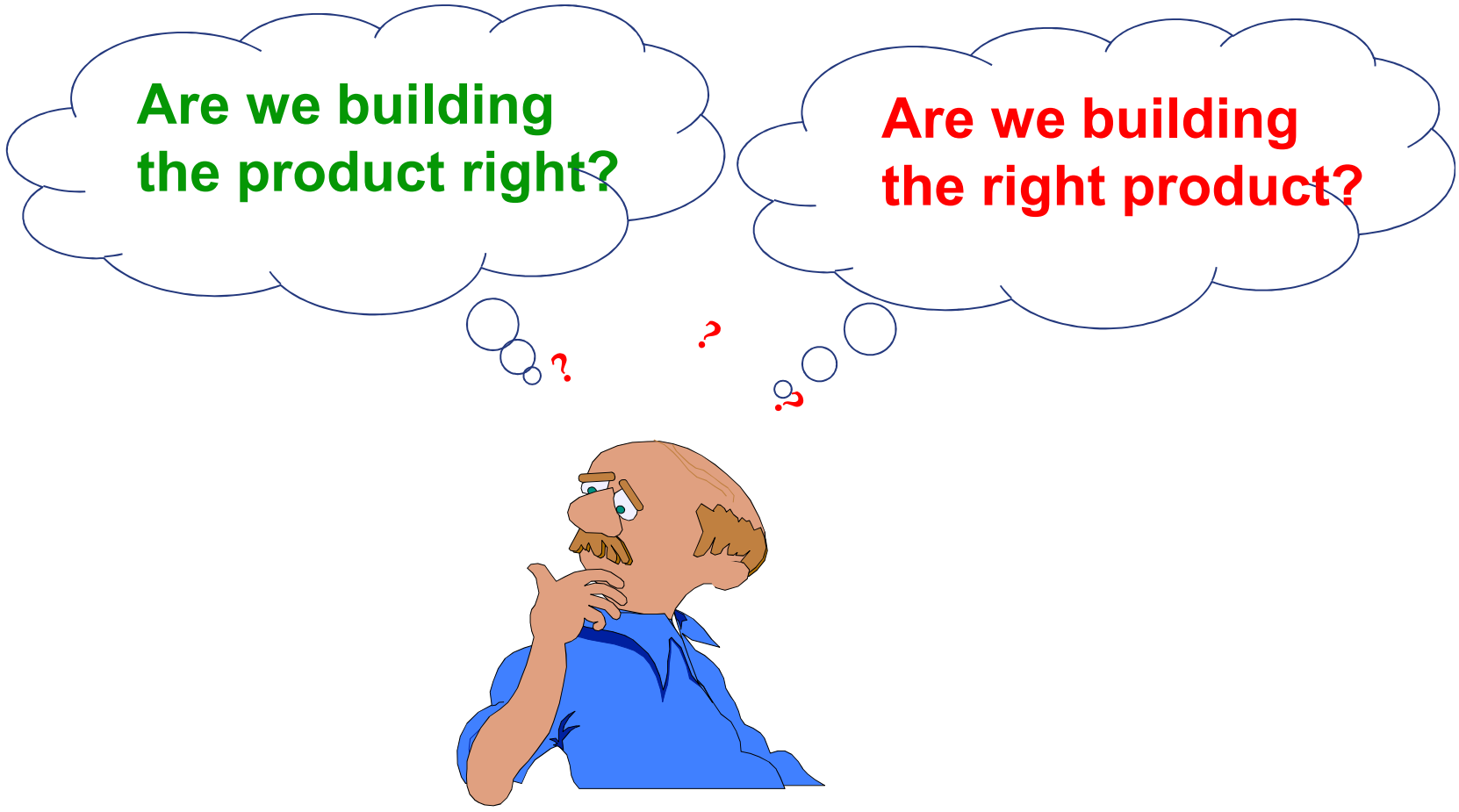
- ***Verification*** **refers to the set of tasks that ensure that software correctly implements a specific function.**

- ***Validation*** **refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.**

3. Verification vs. Validation（角度、标准不同）

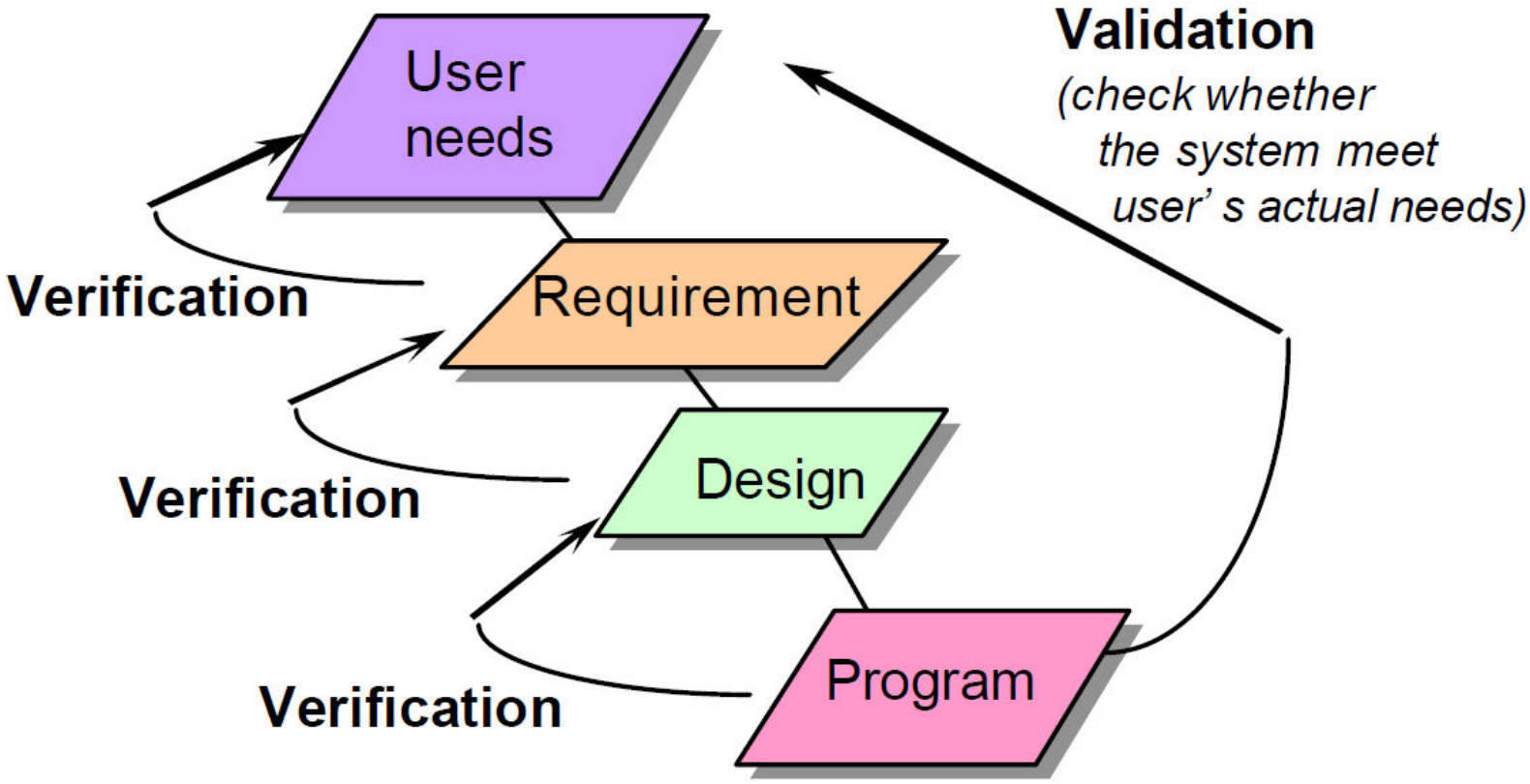
- Verification（局部）——确保软件正确实现特定功能的一组任务
- Validation（整体）——确保已构建的软件可追溯至客户要求

Verification vs. Validation



Boehm [Boe81]

Verification vs. Validation



Summary of V &V

- **Both verification and validation use the same techniques (such as testing, review, inspection, etc.)**
- **The key difference between them is what is used as the "reference for correctness"**
  - **In verification, we use the output from the previous step as the reference of correctness (for example, when we verify a code, we check it against the design)**
  - **In validation, we always use the "user needs" as the reference of correctness (for example, when we validate a code, we check it against the *user needs and expectation*)**

验证和确认（V&V）都使用相同的技术，例如测试、审查、检查等。它们之间的主要区别在于用作"正确性参考"的内容。

验证：

- 使用上一步的输出作为正确性的参考。
- 例如，当我们验证代码时，会根据设计规格进行检查。

确认：

- 总是使用"用户需求"作为正确性的参考。
- 例如，当我们确认代码时，会根据用户需求和期望进行检查。

Testing Strategy

我们从“小测试”开始，然后转向“大测试”。

- We begin by ‘testing-in-the-small’ and move toward ‘testing-in-the-large’
- For conventional software
  - The module (component) is our initial focus
  - Integration of modules follows
- For OO software
  - our focus when “testing in the small” changes from an individual module (the conventional view) to an OO class that encompasses attributes and operations and implies communication and collaboration

对于传统软件：

- 模块（组件）是我们最初关注的焦点。
- 模块集成是随后关注的重点。

对于面向对象软件：

- 当“小测试”从单个模块（传统视角）转变为包含属性和操作的面向对象类时，我们的重点变为类之间的通信和协作。

Testing strategies

- Unit testing
- Integration testing
  - Top down； Bottom up； Regression； Smoke
- System testing
  - Recovery testing； Security testing； Stress testing ； Performance testing
- Validation testing
  - α testing， ß testing

4. 测试策略——从小到大

- 单元测试
- 集成测试
- 系统测试
- 验收测试

Testing Steps

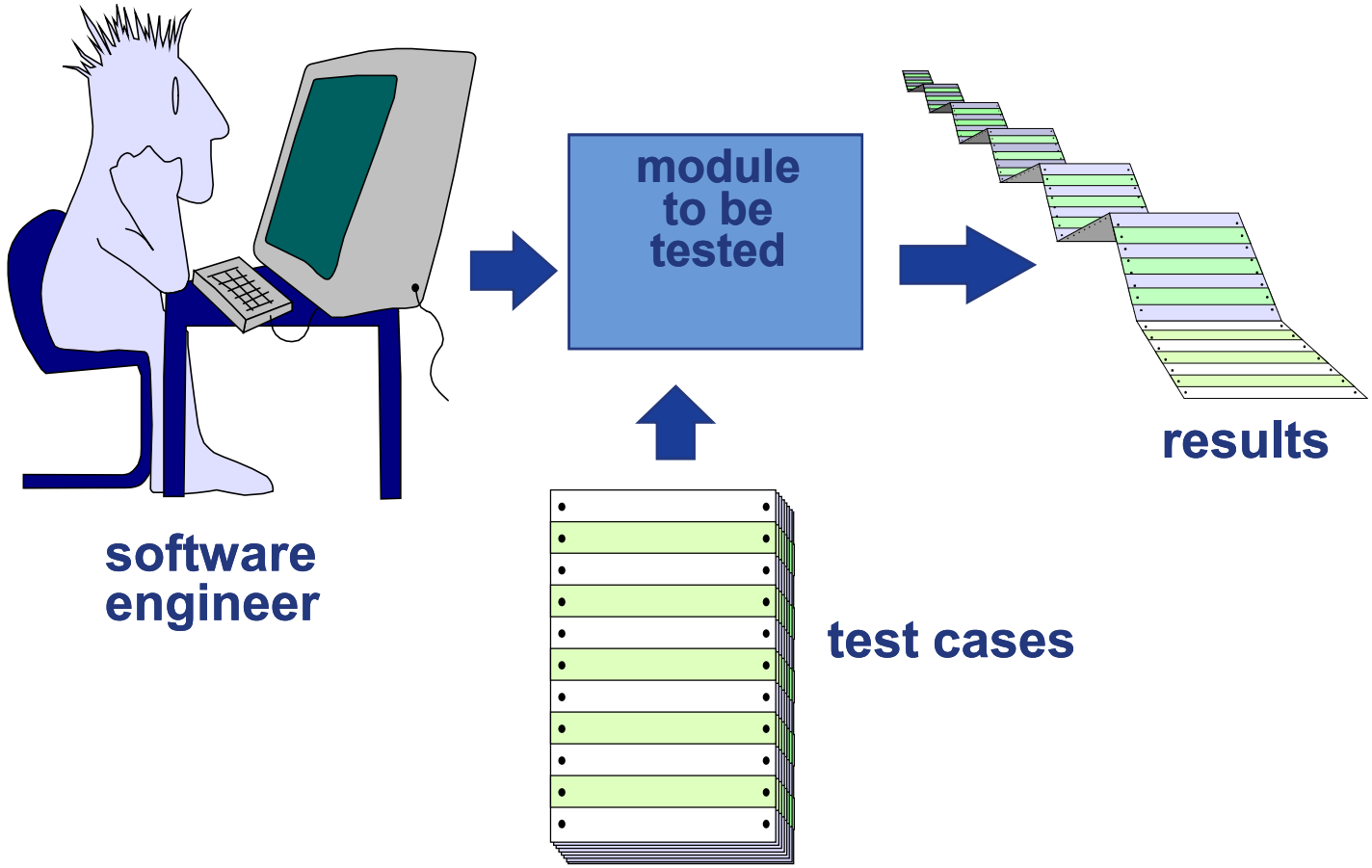
- 1. Before developing a test, we should identify the goals of the test (e.g., reliability, functionality)
- 2. Decide how to carry out a relevant test. We have to decide which test is the most suitable and what sort of test items need to be used.
- 3. Develop the test case.
- 4. Determine the expected results of the test.
- 5. Execute the test cases.
- 6. Compare the test results to the expected results.

Testing Steps

5. 测试的步骤

    1. 在开发测试之前，我们应该确定测试的目标（例如，可靠性、功能性）
    2. 决定如何进行相关测试，决定哪种测试最合适，需要使用什么样的测试项目
    3. 开发测试用例
    4. 确定测试的预期结果
    5. 执行测试用例
    6. 将测试结果与预期结果进行比较

Unit Testing

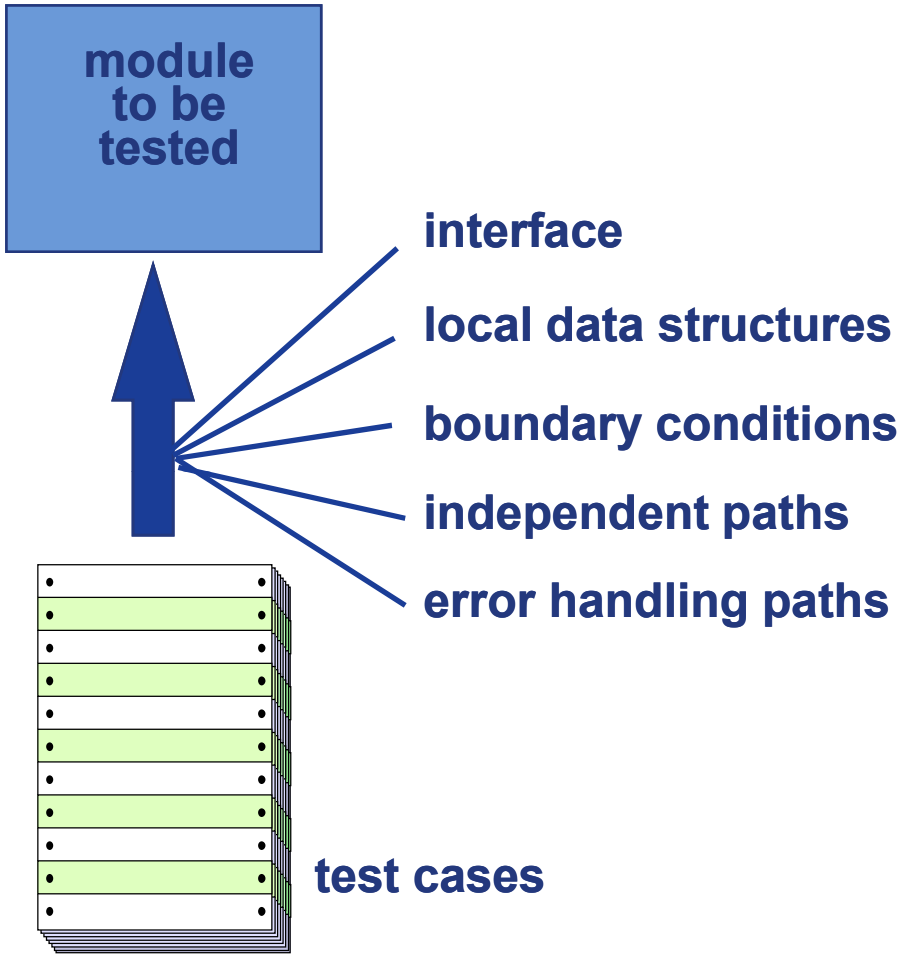


Unit Testing

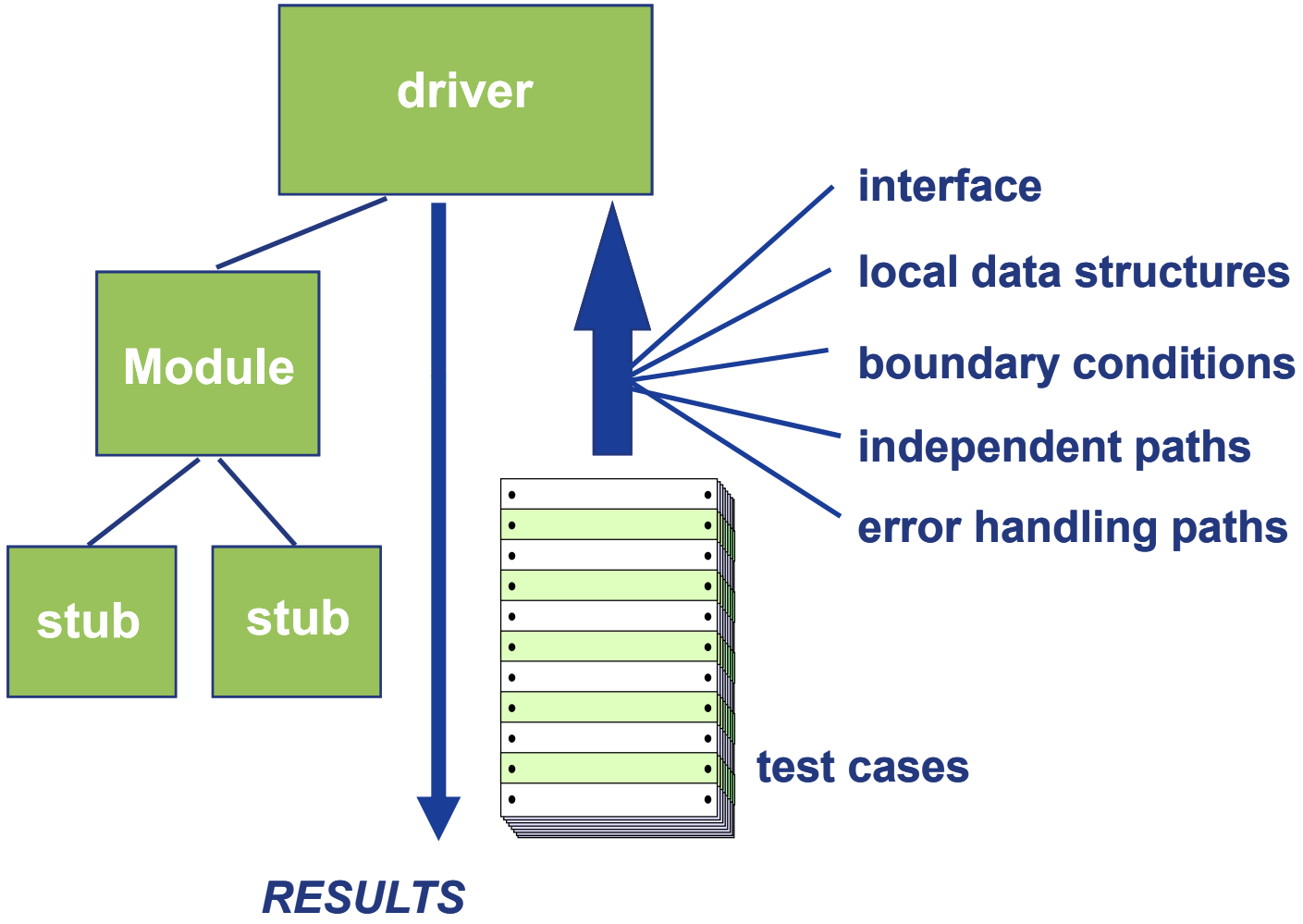
单元测试

测试对象：

- 接口
- 局部数据结构
- 边界条件
- 独立路径
- 错误处理路径



Unit Test Environment



Driver

- **Drivers provide a framework for**
  - **setting input parameters, and the environment**
  - **executing the unit, and**
  - **reading the output parameters (save results and checks they are correct).**
- **Typical Behavior:**
  - **supply a constant input, or a random input**
  - **supply a 'canned' input （屏蔽输入）**
  - **record interim results（中间结果） and traces**

2. driver——代替顶层

- 提供了一个框架用于设置输入参数，环境，执行单元
- 读取输出参数（保存结果并检查它们是否正确）

典型行为：

- 提供恒定输入或随机输入
- 提供"预设"输入
- 记录中间结果和跟踪

Stub

单元测试可能依赖于另一个组件，而该组件可能尚未完成或会在测试中引入不良的副作用。桩模拟单元调用的程序的一部分。

- The unit under test may depend on another component that may not have been completed yet or would introduce undesirable side effects into the test. Stub simulate part of the program called by the unit.
- Example use of Stub:
  - Simulation of target hardware dependent functions
  - Simulation of the functions to be implemented
- Typical behavior:
  - exit immediately
  - return a constant output, or a random output
  - return a value from the user
  - print 'module x entered'

1. stub——代替底层

- 模拟目标硬件相关函数
- 模拟要实现的函数

典型行为：

- 立即退出
- 返回恒定输出或随机输出
- 返回用户的值
- 打印"module x entered"

Why integration testing

- 之前单独测试的逻辑相关单元现在一起进行测试
- 50-75%的错误都是由于接口问题引起的
- 许多接口不兼容性直到集成测试时才会显现出来！！

- **Logically related units that were previously tested separately are tested together**
- **50-75% of all errors are because of interfaces problems.**
- **Many interface incompatibilities don't show up until integration testing!!**
- **Integration testing is to**
  - **Verify that a component work correctly with the rest of the system**
  - **Verify interaction between components**
  - **Check components/subsystems interface (e.g., interface from one menu to other menus): data exchanged between interfaces meets specification**

1. 集成测试的目的

- 验证组件与系统的其余部分是否正常工作
- 验证组件之间的交互
- 检查组件/子系统接口之间交换的数据是否符合规范

Integration Testing Strategies

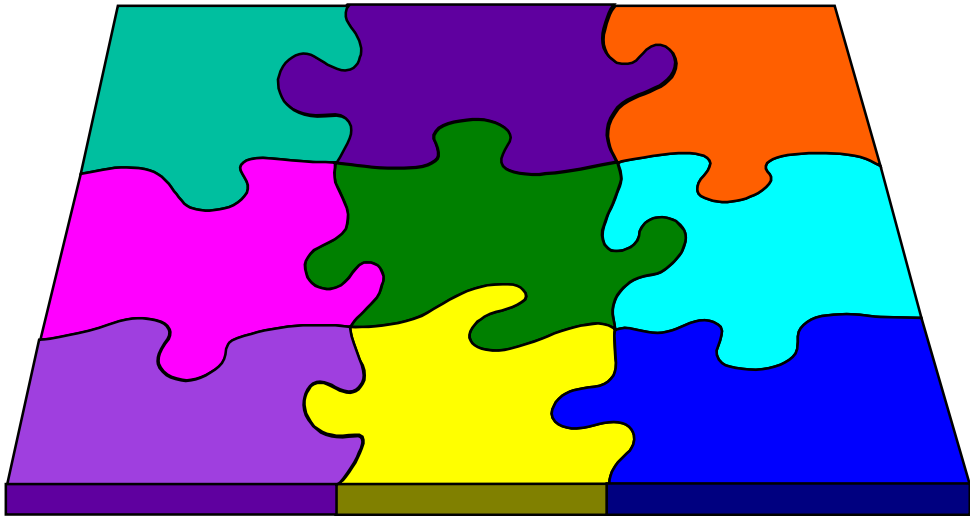
Options:

- **the “big bang” approach**
- **an incremental construction strategy**

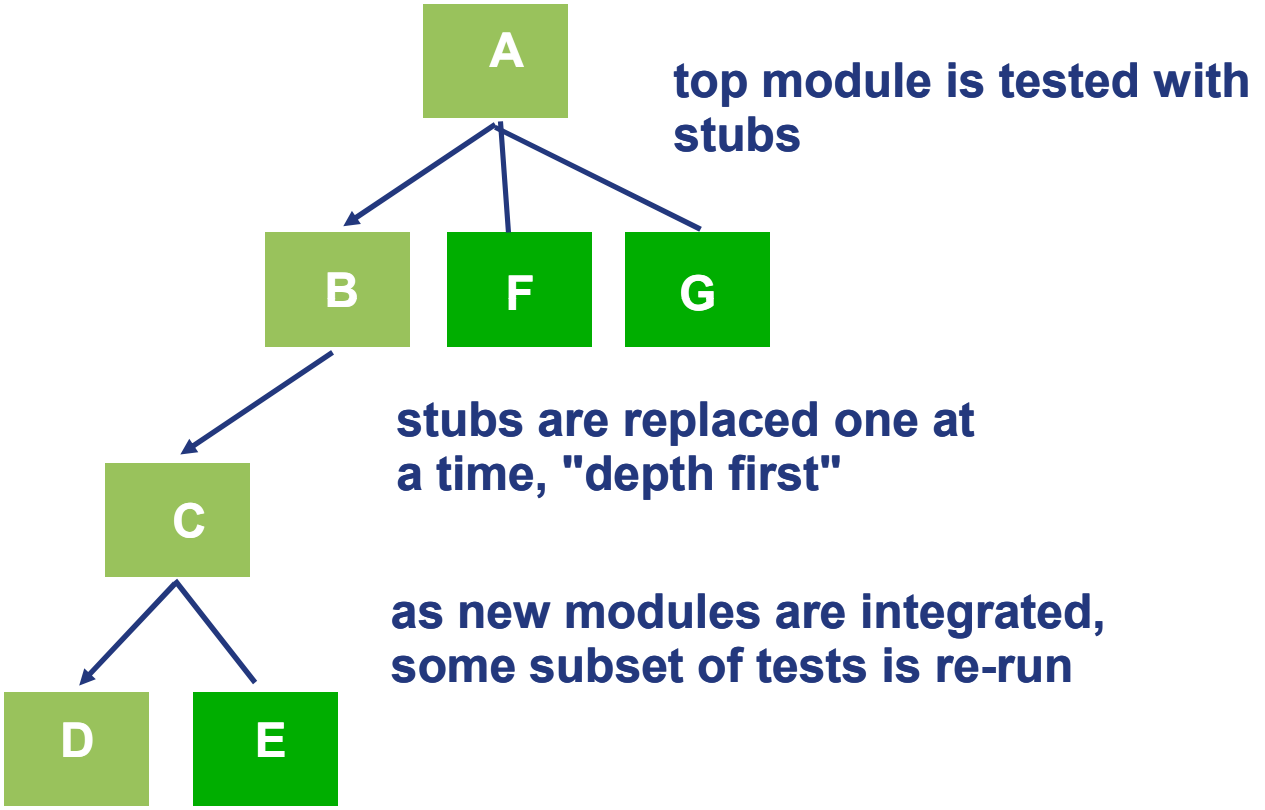
2. 集成测试的策略

1. Big Bang

2. 增量集成



Top Down Integration



2. 增量集成

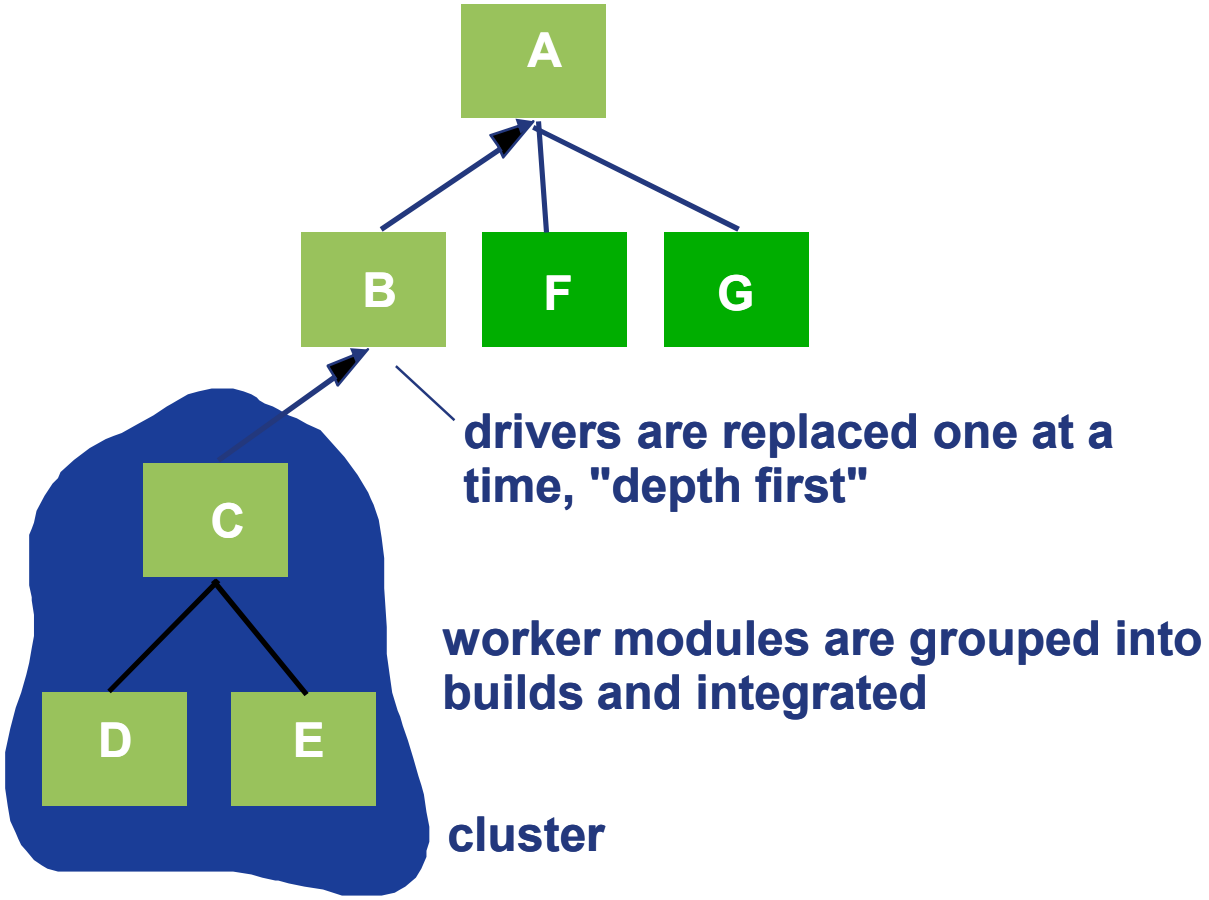
- **自顶向下**（深度或者广度优先）

Top Down Integration

- **Advantage:**
  - **A skeletal version of the program can exist early and allows demonstrations.**
  - **Design errors may be found sooner.**
  - **Reduces the need for test drivers.**
  - **It tends to make fault location easier.**
- **Disadvantage:**
  - **stubs could be expensive to build.**

- 优势
  - 程序的骨架版本可以早期存在并允许演示
  - 设计错误可能会更快被发现
  - 减少对测试驱动程序的需求
  - 故障定位更容易
- 缺点
  - stub的构建成本可能很高

Bottom-Up Integration



Bottom-Up Integration

Advantages:

- **Test cases for bottom up testing may be designed solely from functional design information, requiring no structural design information. The bottom –up approach is useful when the detailed design documentation lacks structural detail.**
- **Particular useful for objects and reuse.**

- 优势
  - 自下而上测试的测试用例可以仅根据功能设计信息设计，不需要结构设计信息。当详细的设计文档缺乏结构细节时，自下而上的方法很有用
  - 特别适用于对象和重用。

Bottom-Up Integration

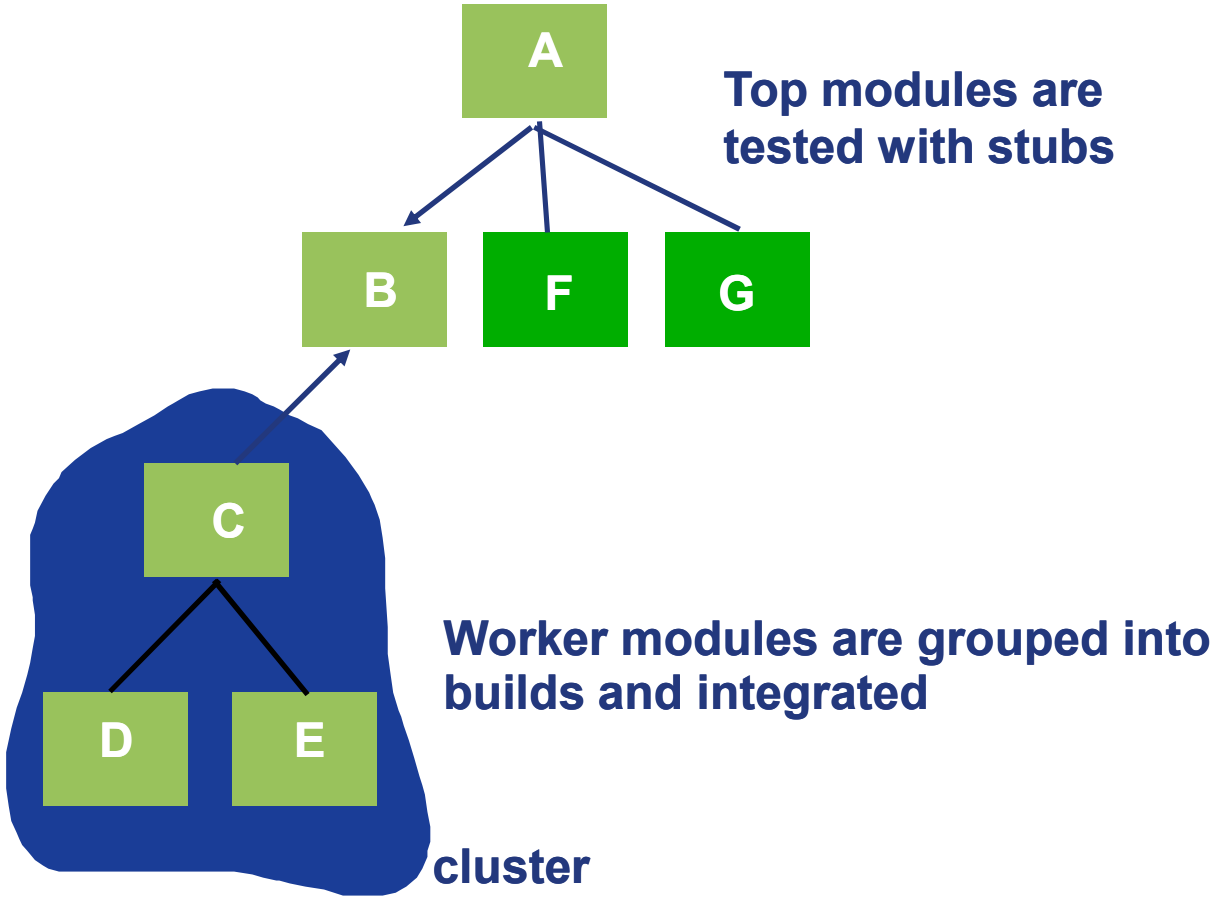
Disadvantages:

- **The program as a whole does not exist until the last module is added.**
- **Bottom up testing requires test drivers, not test stubs.**
- **As testing progresses up the hierarchy, bottom up testing becomes more complicated, and consequently more expensive to develop and maintain; it becomes more difficult to achieve good structural coverage.**

- 缺点
  - 在添加最后一个模块之前，程序作为一个整体是不存在的
  - 自下而上的测试需要测试driver，而不是测试stub
  - 随着测试在层次结构上进行，自下而上的测试变得更加复杂，因此开发和维护成本更高；实现良好的结构覆盖变得更加困难

Sandwich Testing

- 三明治测试——工作模块被集合成cluster变成大模块



System testing

- System testing is a series of different tests whose primary purpose is to fully exercise the computer-based system
- Test the complete system in the real environment in which it is to operate
- Ensure system requirements are met “Does the system works as a whole?”
- Also test areas other than just the functioning of the system
- Results are sometimes used for system acceptance
- Verify the Software User Manual
- Estimate reliability and maintainability

系统测试的目的

- 在将要运行的真实环境中测试整个系统
- 确保系统满足整体工作的要求
- 还要测试系统功能之外的方面
- 结果有时用于系统验收
- 验证软件用户手册
- 估计可靠性和可维护性

Specific Testing

- **WebApp Testing**
  - **Test the contents，interfaces，navigation mechanics，functional components**
  - **Test the implementation in different environmental configurations，the security，reliability and performance.**
- **MobileApp Testing**
  - **Test the usability and accessibility ；**
  - **Test on multiple devices，in actual user environments；**
  - **Test the security，reliability and performance.**
  - **meets the distribution standards**

Web应用测试

- 测试内容、界面、导航机制、功能组件
- 测试在不同环境配置下的实现情况，以及安全性、可靠性和性能

移动应用测试

- 测试可用性和可访问性
- 在多种设备和实际用户环境中进行测试
- 测试安全性、可靠性和性能
- 符合分发标准

Validation testing

Validation testing

- **Focus is on software requirements**
- **After validation testing**
  - **The function or performance conforms to specification.**
  - **Use a deficiency list to describe the deviation（偏差） from specification**
- **Alpha/Beta testing**

验收测试——关注软件需求

产出

- 功能或性能符合规范
- 使用缺陷列表来描述与规范的偏差

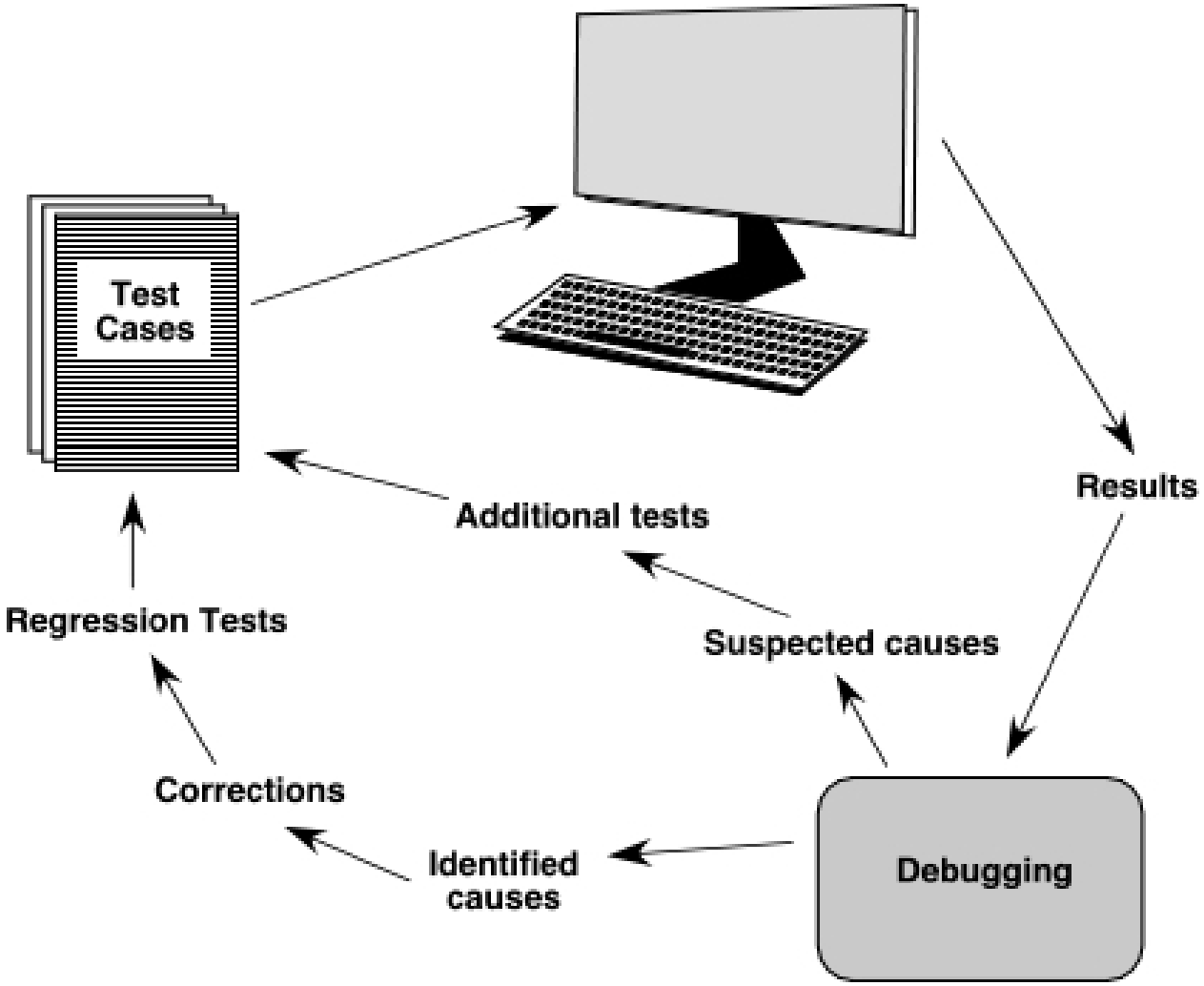
Alpha/Beta测试——关注客户使用情况

Alpha 测试由最终用户在开发人员站点进行

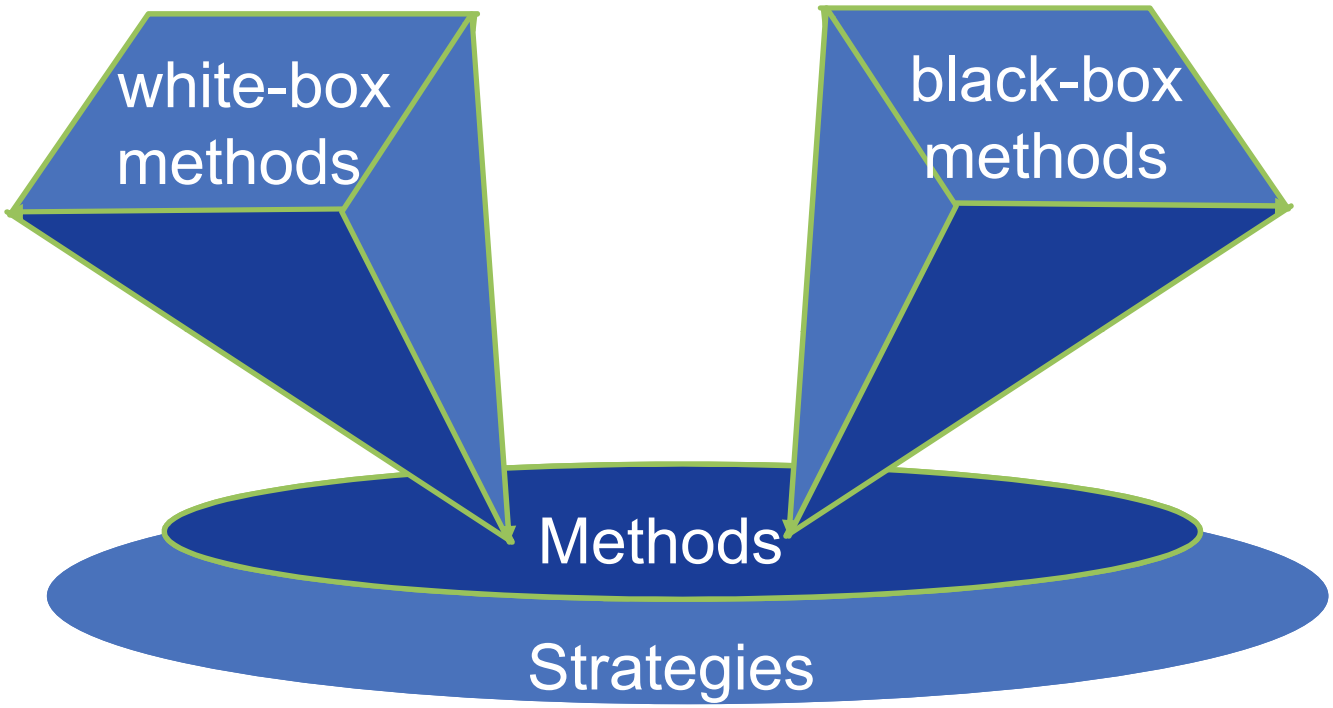
Beta 测试在最终用户站点进行

Debugging: A Diagnostic Process

The Debugging Process



Software Testing Techniques



Software Testing Techniques

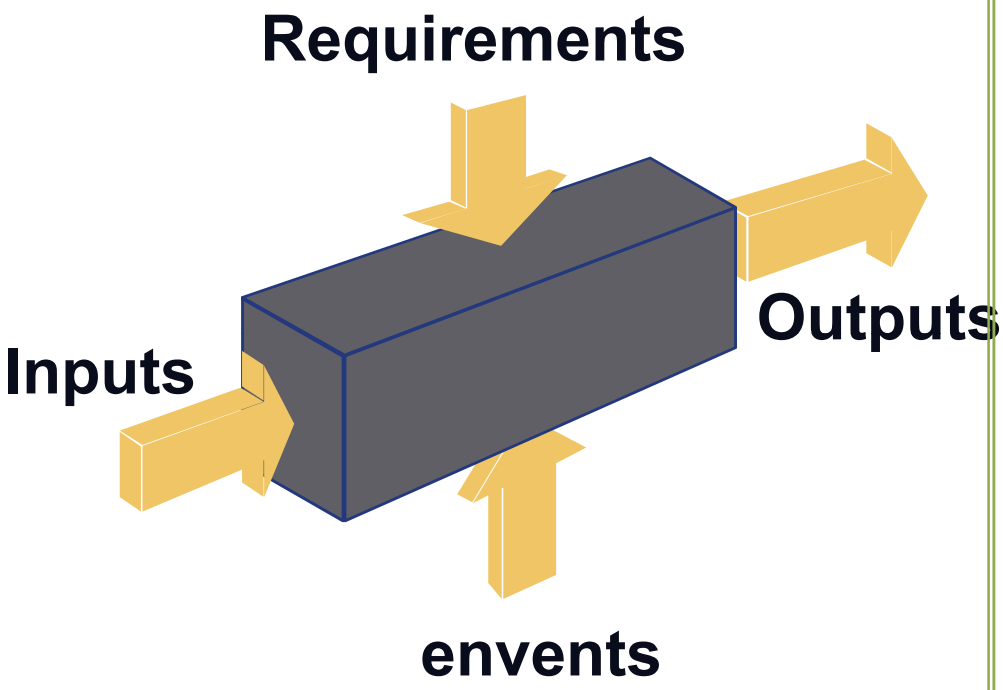
1. 黑盒测试（功能化）
   - 用例基于软件规约
   - 可能会发现与白盒测试不同的错误类别
   - 倾向于在测试的后期阶段应用
   - 查找的错误
     - 不正确或缺少函数
     - 接口错误
     - 数据结构或外部数据库访问错误
     - 行为或性能错误
     - 初始化和终止错误

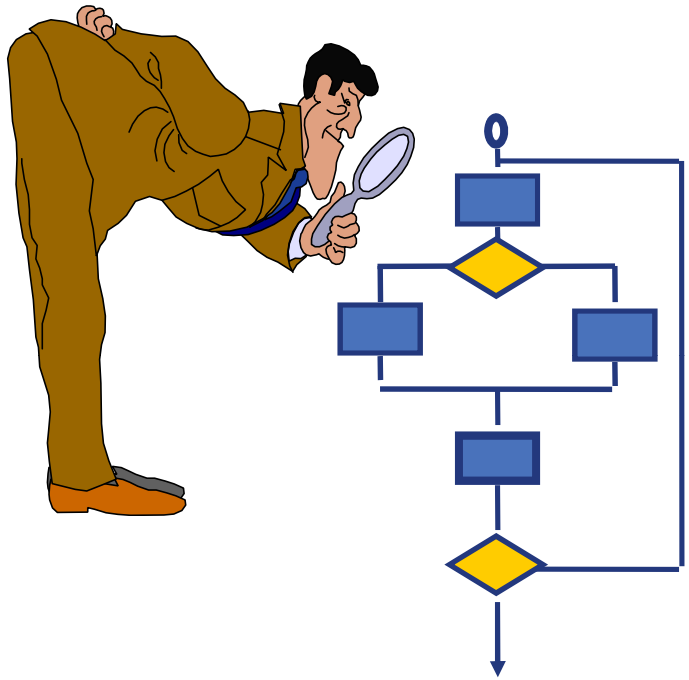
2. 白盒测试（结构化）
   - 用例基于代码
   - 为什么要结构化测试

     避免没测试到的部分产生问题
     覆盖低层次的细节

   - 执行方式
     - 保证模块内的所有独立路径至少被执行一次
     - 执行所有逻辑决策的true和false
     - 在其边界和操作范围内执行所有循环界限
     - 检验内部数据结构以确保其有效性

Software Testing Techniques





Testing Tactics

- Tests based on *spec*
- Test covers as much *specified* behavior as possible

- Tests based on *code*
- Test covers as much *implemented* behavior as possible

Why Structural?

- If a part of the program is never executed, a defect may loom in that part
  A "part" can be a statement, function, transition. condition…
- Attractive because automated

Why Structural?

- Complements functional tests
  Run functional tests first, then measure what is missing
- Can cover low-level details missed in high-level specification

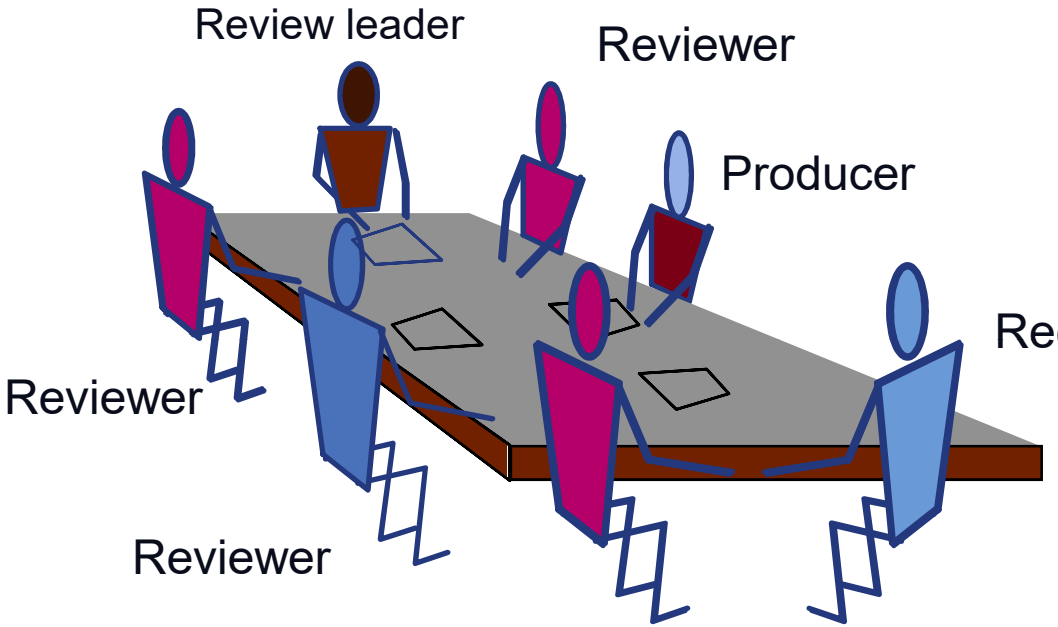
Black Box vs. White Box testing

Black Box Methods	White Box Methods
Equivalence Partitioning （等价类划分）	Desk Checking （桌面检查）
Boundary Value Analysis （边界值分析）	Walkthrough （代码走查）
Graph-Based Methods （因果图分析）	Inspection （代码审查）
Orthogonal Array Testing （正交数组法）	Basis Path Testing （基本路径测试）
……	……

Static Testing

Software Testing Techniques

Static Testing



Informal — formal

Desk checking **walkthrough** **inspection**

Dynamic Testing

Run the software

4. 静态测试和动态测试

- 静态测试——检查
- 动态测试——运行

Chap 17-19. Software Testing Strategies and Techniques Review

- 1. Concepts
- 2. V model vs. W model
  - Verification & Validation
- 3. Testing Strategies
  - Unit testing
  - Integration testing
  - System testing
  - Validation testing
  - Testing vs. Debug
- 4. Testing techniques
  - White box vs. Black box
  - Static testing vs. Dynamic testing

Southeast University

Thank You !

lliao@seu.edu.cn